AI ASSISTED CODING

ASSIGNMENT-13

B. Sri Laxmi Gayathri

2303a52033

Batch:38

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

• **Task:** Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions.

Sample Code:

```
length = 8

width = 4

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))

length = 12

width = 6

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Prompt:

Use AI-assisted refactoring to identify repeated calculations in the given Python code and convert them into a reusable function that computes both area and perimeter of a rectangle.

Code & Output:

```python
# This code calculates the area and perimeter of a rectangle given its length and width.
def calculate_area_perimeter(length, width):
    """
    Calculates the area and perimeter of a rectangle.
    Args:
        length (int or float): The length of the rectangle.
        width (int or float): The width of the rectangle.
    Returns:
        dict: A dictionary containing 'Area' and 'Perimeter'.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return {"Area": area, "Perimeter": perimeter}
# First set of dimensions
length_1 = 8
width_1 = 4
results_1 = calculate_area_perimeter(length_1, width_1)
print("Length:", length_1, ", Width:", width_1)
print("Area:", results_1["Area"])
print("Perimeter:", results_1["Perimeter"])
print("\n")
# Second set of dimensions
length_2 = 12
```

```python
width_2 = 6
results_2 = calculate_area_perimeter(length_2, width_2)
print("Length:", length_2, ", Width:", width_2)
print("Area:", results_2["Area"])
print("Perimeter:", results_2["Perimeter"])
```

```
Length: 8 , Width: 4
Area: 32
Perimeter: 24


Length: 12 , Width: 6
Area: 72
Perimeter: 36
```

Explanation:

The AI analyzes the program and detects that the area and perimeter calculations are repeated for different rectangle dimensions. To improve maintainability, the repeated logic is placed inside a single reusable function. This function accepts length and width as parameters and calculates both values.

Task Description -2 (Refactoring – Modularizing Inline Calculations)

• **Task:** Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

Sample Code:

```
amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)
```

Expected Output:

A modular Python program that uses a reusable function to compute final amounts for different inputs.

Prompt:

Use AI-assisted refactoring to analyze the Python code and identify repeated discount calculations. Convert the repeated inline logic into a reusable function with proper docstrings while keeping the program output the same.

Code & Output:

```python
def calculate_final_amount(amount, discount_rate):
    """
    Calculates the final amount after applying a discount.
    Args:
        amount (float or int): The initial amount.
        discount_rate (float): The discount rate (e.g., 0.10 for 10%).
    Returns:
        float: The final amount after the discount.
    """
    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount
# First set of calculations
amount_1 = 1000
discount_rate = 0.10
final_amount_1 = calculate_final_amount(amount_1, discount_rate)
print(f"Initial Amount: {amount_1}, Discount Rate: {discount_rate*100}% ")
print("Final Amount:", final_amount_1)
print("\n")
# Second set of calculations
amount_2 = 2000
final_amount_2 = calculate_final_amount(amount_2, discount_rate)
```

```python
print(f"Initial Amount: {amount_2}, Discount Rate: {discount_rate*100}% ")
print("Final Amount:", final_amount_2)
```

```
Initial Amount: 1000, Discount Rate: 10.0%
Final Amount: 900.0


Initial Amount: 2000, Discount Rate: 10.0%
Final Amount: 1800.0
```

Explanation:

The AI examines the script and finds that the discount and final amount calculations are repeated for different values. To improve modularity, the repeated logic is extracted into a reusable function that takes the amount as input and returns the final amount after applying the discount. This reduces code duplication and improves readability.

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

• **Task:** Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

• Use AI to recommend more efficient data structures or logic simplifications.

• Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]

check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:

exists = False

for student in students:

if name == student:

exists = True

if exists:

print(name, "found")

else:

print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

Prompt:

Use AI to analyze the Python code with nested loops and repeated conditional checks, then refactor it by suggesting a more efficient data structure and simplified logic.

Code & Output:

```python
#loop and conditional statements in python
#1. Looping through a list of numbers and printing their squares
numbers = [1, 2, 3, 4, 5]
print("Squares of the numbers:")
for num in numbers:
    print(num ** 2)
print("\n")
#2. Using a conditional statement to check if a number is even or odd
number = 10
if number % 2 == 0:
    print(f"{number} is an even number.")
else:
    print(f"{number} is an odd number.")
print("\n")
#3. Looping through a list of names and printing a greeting for each name
names = ["Alice", "Bob", "Charlie"]
print("Greetings:")
for name in names:
    print(f"Hello, {name}!")
print("\n")
#4. Using a conditional statement to check if a number is positive, negative, or zero
number = -5
if number > 0:
```

```python
    print(f"{number} is a positive number.")
elif number < 0:
    print(f"{number} is a negative number.")
else:
    print(f"{number} is zero.")
```

```
Squares of the numbers:
1
4
9
16
25


10 is an even number.
```

```
Greetings:
Hello, Alice!
Hello, Bob!
Hello, Charlie!


-5 is a negative number.
```

Explanation:

The AI identifies that the nested loop used to search for student names is inefficient because it checks every element repeatedly. To improve performance, the list of students can be converted into a set, which allows faster membership checks.

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

• **Task:** Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing results.

Sample Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..

Prompt:

Use AI-assisted refactoring to identify hardcoded numeric values in the Python code and replace them with descriptive constants. Ensure the calculations remain the same while improving readability and maintainability.

Code & Output:

```python
# Define constants
RATE = 5      # Annual interest rate in percentage
TIME = 2      # Time period in years
PERCENTAGE_DIVISOR = 100 # Divisor to convert percentage to decimal
def calculate_simple_interest(principal, rate, time, divisor):
    """
    Calculates the simple interest.
    Args:
        principal (int or float): The principal amount.
        rate (int or float): The annual interest rate in percentage.
        time (int or float): The time period in years.
        divisor (int or float): The divisor to convert percentage to decimal (usually 100).
    Returns:
        float: The calculated simple interest.
    """
    return (principal * rate * time) / divisor
# First calculation
principal_1 = 1000
simple_interest_1 = calculate_simple_interest(principal_1, RATE, TIME, PERCENTAGE_DIVISOR)
print(f"Principal: {principal_1}, Simple Interest: {simple_interest_1}")
# Second calculation
principal_2 = 2000
simple_interest_2 = calculate_simple_interest(principal_2, RATE, TIME, PERCENTAGE_DIVISOR)
print(f"Principal: {principal_2}, Simple Interest: {simple_interest_2}")
```

```
Principal: 1000, Simple Interest: 100.0
Principal: 2000, Simple Interest: 200.0
```

Explanation:

The AI analyzes the code and detects that numeric values like the interest rate and time are directly used in the formula. These values are replaced with named constants such as RATE and TIME defined at the beginning of the program. Using constants makes the code easier to understand and modify in the future. This refactoring improves maintainability without changing the program's output.

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

• **Task:** Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

• Replace non-descriptive variable names with meaningful identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same.

Sample Code:

x = 50

y = 30

z = x + y

print(z)

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

Prompt:

Use AI-assisted refactoring to improve the readability of the Python code by replacing unclear variable names with meaningful identifiers and adding helpful inline comments, while ensuring the program output remains unchanged.

Code & Output:

```
#variable name and code clarity in python
#1. Using descriptive variable names for better code clarity
# Instead of using generic variable names like 'x' and 'y', we can use descriptive
# variable names to make the code more readable and understandable.
# Example of using descriptive variable names
length_of_rectangle = 10
width_of_rectangle = 5
area_of_rectangle = length_of_rectangle * width_of_rectangle
print("Length of Rectangle:", length_of_rectangle)
print("Width of Rectangle:", width_of_rectangle)
print("Area of Rectangle:", area_of_rectangle)
print("\n")
#2. Using comments to explain the purpose of code blocks
# Comments can help explain the purpose of different code blocks and make it easier for others (or
# yourself in the future) to understand the logic behind the code.
# Example of using comments to explain code blocks
# Function to calculate the area of a circle
def calculate_circle_area(radius):
    """
    Calculates the area of a circle given its radius.
    Args:
        radius (float or int): The radius of the circle.
    Returns:
```

```python
        float: The area of the circle.
    """
    import math
    area = math.pi * (radius ** 2)
    return area
# Using the function to calculate the area of a circle with radius 7
radius_of_circle = 7
area_of_circle = calculate_circle_area(radius_of_circle)
print("Radius of Circle:", radius_of_circle)
print("Area of Circle:", area_of_circle)
print("\n")
#3. Avoiding ambiguous variable names
# Using ambiguous variable names can lead to confusion and make the code harder to understand.
# Example of avoiding ambiguous variable names
# Instead of using 'data' as a variable name, we can use 'user_data' to clarify what the data represents.
user_data = {
    "name": "John Doe",
    "age": 30,
    "email": "john.doe@example.com"
}
print("User Data:", user_data)
```

```
Length of Rectangle: 10
Width of Rectangle: 5
Area of Rectangle: 50


Radius of Circle: 7
Area of Circle: 153.93804002589985


User Data: {'name': 'John Doe', 'age': 30, 'email': 'john.doe@example.com'}
```

Explanation:

The AI reviews the code and identifies that variables like x, y, and z do not clearly describe their purpose. These variables are renamed to more meaningful names such as num1, num2, and total_sum to improve clarity. Additionally, inline comments are added to explain the logic of the calculation.